

Pontificia Universidad Católica del Perú
Maestría en Ingeniería de Control y Automatización

Redes Neuronales Aplicadas al Control

Resumen de estudio del curso (Clases 7–13)

Material consolidado por temas, sin repeticiones. Cada sección indica la(s) clase(s) de origen.

Contenido

1	Modelamiento de sistemas dinámicos con redes neuronales	[Clase 7]	2
1.1	Modelo estático vs. modelo dinámico		2
1.2	Modelamiento con redes estáticas (dinámica artificial)		2
1.3	Modelamiento con redes dinámicas (recurrentes)		2
2	Control de sistemas usando redes neuronales	[Clase 9]	3
2.1	Objetivos de control		3
2.2	Aprendizaje estático		3
2.3	Aprendizaje dinámico y despliegue temporal		4
2.4	El algoritmo DBP para control (fórmula central del curso)		4
2.5	Buenas prácticas de entrenamiento		4
2.6	Tracking y realimentación parcial		4
2.7	Integración de sistemas lineales y redes neuronales		5
3	Robots móviles tipo carro: modelamiento y control	[Clase 10]	5
3.1	Tipos y modelo cinemático (modelo bicicleta)		5
3.2	Estrategia de posicionamiento		5
3.3	Seguimiento de trayectorias: posición deseada perpendicular		5
4	Control difuso: fundamentos y diseño	[Clases 10 y 11 — material común]	6
4.1	Motivación y conceptos básicos		6
4.2	Proceso de diseño de un controlador difuso (receta general)		6
4.3	Ejemplo canónico: robot carro (recta vertical $x^* = 50$, $\phi^* = 90^\circ$)		7
4.4	Segundo ejemplo: posición de un motor DC con tornillo sinfín		7
5	Control difuso del robot truck-tráiler	[Clase 12]	8
6	Redes neuro-difusas	[Clase 12]	8
6.1	Motivación y tipos		8
6.2	Estructura (red Tipo 1) y entrenamiento		8
6.3	Formación de robots (aplicación)		9
7	Control lineal-difuso del truck-tráiler	[Clase 13 — paper Morán & Nagai]	9
7.1	Modelo y estrategia		9
7.2	Diseño: linealización + integración difusa		9

7.3	Seguimiento de trayectorias (posición deseada perpendicular)	9
7.4	Efecto del ancho de las particiones de θ_{12}	10
8	Formulario / hoja de repaso rápido	11

1 Modelamiento de sistemas dinámicos con redes neuronales [Clase 7]

1.1 Modelo estático vs. modelo dinámico

Una red neuronal puede entrenarse como **modelo** de un sistema dinámico a partir de datos entrada-salida (u_k, y_{k+1}^*) . La distinción fundamental es *de dónde proviene la entrada y_k del modelo*:

	Modelo estático	Modelo dinámico (recurrente)
Ecuación	$y_{k+1} = f(y_k^*, u_k)$	$y_{k+1} = f(y_k, u_k)$
Entrada y_k	medida del sistema real	realimentada del propio modelo
¿Requiere el sistema en operación?	sí (inconveniente para simular y predecir)	no (tras entrenar es independiente)
Robustez al ruido	baja (el ruido entra a la red)	alta (el ruido sólo afecta al entrenamiento)
Entrenamiento	backpropagation estándar	BPTT o DBP (derivadas totales)

1.2 Modelamiento con redes estáticas (dinámica artificial)

Se crea una dinámica artificial usando como entradas de la red **valores pasados** de u y de y^* :

$$y_{k+1} = f(y_k^*, y_{k-1}^*, \dots, u_k, u_{k-1}, \dots)$$

El número de retardos se elige por prueba y error, o analizando el **modelo lineal aproximado**: si

$$H(z) = \frac{Y(z)}{U(z)} = \frac{b_1 z + b_0}{z^2 + a_1 z + a_0} \implies y_{k+1} = -a_1 y_k - a_0 y_{k-1} + b_1 u_k + b_0 u_{k-1}$$

un primer intento de entradas es $\{y_k, y_{k-1}, u_k, u_{k-1}\}$. Se generaliza a sistemas MIMO usando los pasados de cada entrada y cada salida. Normalmente no se requiere neurona de sesgo (equivale a un término constante en la ecuación del sistema).

1.3 Modelamiento con redes dinámicas (recurrentes)

La salida de la red se realimenta como entrada en el paso siguiente. Con datos $\{u_k, y_k^*\}_{k=1}^N$ se minimiza

$$J = \frac{1}{2} \sum_{k=1}^N (y_k - y_k^*)^T (y_k - y_k^*), \quad v \leftarrow v - \eta \frac{\partial J}{\partial v}, \quad w \leftarrow w - \eta \frac{\partial J}{\partial w}$$

donde las derivadas son **totales** porque y_k depende de los pesos a través de todos los pasos anteriores:

$$\frac{\partial J}{\partial w} = \sum_{k=1}^N (y_k - y_k^*)^T \frac{\partial y_k}{\partial w}$$

Ecuación recursiva (DBP) para modelamiento — la misma idea reaparece en control:

$$\frac{\partial y_{k+1}}{\partial w} = \frac{\partial y_{k+1}}{\partial w} + \frac{\partial y_{k+1}}{\partial y_k} \frac{\partial y_k}{\partial w}, \quad \frac{\partial y_0}{\partial w} = 0$$

donde $\frac{\partial y_{k+1}}{\partial w}$ (derivada simple) y el Jacobiano $\frac{\partial y_{k+1}}{\partial y_k}$ se calculan directamente de la red en el último paso, pues todas sus ecuaciones son conocidas. Se prefiere DBP sobre BPTT por eficiencia de memoria (avanza hacia adelante junto con la simulación).

Medición parcial. Si el estado es $[y_1 \ y_2]^T$ pero sólo se mide y_1 , el costo se define sólo con la variable medida usando la matriz de ponderación:

$$J = \frac{1}{2} \sum_k (y_k - y_k^*)^T Q (y_k - y_k^*), \quad Q = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}$$

y las derivadas totales se calculan igual, con la misma recursión.

Identificación de parámetros de sistemas lineales. Un sistema lineal $y_{k+1} = Ay_k + Bu_k$ se representa con una red de *neuronas lineales* cuyos pesos son directamente a_{ij} y b_i ; el entrenamiento identifica los parámetros. (Para sistemas lineales existe realización mínima controlable y observable.)

Riqueza de datos. La entrada u_k de entrenamiento debe ser rica y variada para excitar toda la dinámica del sistema.

2 Control de sistemas usando redes neuronales

[Clase 9]

2.1 Objetivos de control

- **Estabilización:** hacer BIBO/asintóticamente estable un sistema inestable (converger a un equilibrio).
- **Regulación:** mantener un punto de operación pese a perturbaciones internas/externas (ej.: aislamiento de vibraciones).
- **Tracking:** seguir una trayectoria de referencia.

Para sistemas inestables primero se resuelve la estabilización y luego se adapta el controlador a regulación/tracking.

El neuro-controlador considerado es una red *estática* $u_k = N(y_k, r)$; aun así, **el lazo cerrado es dinámico** (la salida se vuelve entrada del paso siguiente). Ambos métodos de entrenamiento son *basados en modelo* (se requiere $y_{k+1} = f(y_k, u_k)$, como ecuación o como red).

2.2 Aprendizaje estático

La red se entrena para que la integración controlador-sistema en **lazo abierto** tenga el mismo mapeo entrada-salida que un *sistema deseado* $y_{k+1}^d = A_d y_k$. Costo (con ponderaciones Q, R):

$$J = \frac{1}{2} \sum_k \left[(y_k - y_k^d)^T Q (y_k - y_k^d) + u_k^T R u_k \right]$$

Se usan **derivadas parciales simples** (no se considera la dinámica del lazo):

$$\frac{\partial J_k}{\partial w} = (y_{k+1} - y_{k+1}^d)^T \frac{\partial y_{k+1}}{\partial u_k} \frac{\partial u_k}{\partial w}$$

donde $\frac{\partial y_{k+1}}{\partial u_k}$ sale del *modelo* y $\frac{\partial u_k}{\partial w}$, $\frac{\partial u_k}{\partial v}$ del *controlador*. Puede entrenarse patrón a patrón (pattern) o por lotes (batch), y el controlador puede incluir salidas pasadas como entradas.

Limitaciones del aprendizaje estático: (a) el costo no siempre decrece: hay que monitorear y detener el entrenamiento; (b) se aplica sobre todo a sistemas *estables* (mejorar respuesta, regulación, tracking); (c) suele fallar con sistemas *inestables*, porque ignora la dinámica del lazo cerrado. Para inestables: aprendizaje dinámico.

2.3 Aprendizaje dinámico y despliegue temporal

La red se entrena para que el **lazo cerrado** tenga la respuesta de un sistema deseado $y_{k+1}^d = A_d y_k^d$ (elegido físicamente alcanzable). Al desplegar el lazo en el tiempo:

- El mismo lazo (misma red, mismos pesos v, w ; mismo modelo) se repite en cada paso $1, 2, \dots, N$.
- La salida del paso k es entrada del paso $k + 1$.
- Aunque w es el mismo en todos los pasos, la *derivada* de y_k respecto al w de cada paso es distinta; la **derivada total** $\frac{\partial y_k}{\partial w}$ comprende la dependencia por todos los caminos.

Los pesos se actualizan con derivadas totales: $\frac{\partial J}{\partial w} = \sum_k (y_k - y_k^d)^T Q \frac{\partial y_k}{\partial w}$ (ídem v).

2.4 El algoritmo DBP para control (fórmula central del curso)

Ecuación recursiva del DBP para el lazo cerrado $u_k = N(y_k; \theta)$, $y_{k+1} = f(y_k, u_k)$, con $\theta \in \{v, c, a, w\}$:

$$\frac{\partial y_{k+1}}{\partial \theta} = \underbrace{\frac{\partial y_{k+1}}{\partial u_k} \frac{\partial u_k}{\partial \theta}}_{\text{aporte del paso presente}} + \underbrace{\left[\frac{\partial y_{k+1}}{\partial y_k} + \frac{\partial y_{k+1}}{\partial u_k} \frac{\partial u_k}{\partial y_k} \right]}_{\text{Jacobiano del lazo cerrado}} \frac{\partial y_k}{\partial \theta}, \quad \frac{\partial y_0}{\partial \theta} = 0$$

Con el **modelo del sistema** se calculan: $\frac{\partial y_{k+1}}{\partial y_k}$ (Jacobiano) y $\frac{\partial y_{k+1}}{\partial u_k}$.

Con el **neuro-controlador** se calculan: $\frac{\partial u_k}{\partial \theta}$ (derivadas simples respecto a pesos/parámetros) y $\frac{\partial u_k}{\partial y_k}$.

Frente al BPTT (que retropropaga por toda la trayectoria almacenada), el DBP avanza *hacia adelante* junto con la simulación y es más eficiente en memoria.

Para la red de una capa oculta con **sigmoide bipolar** (la usada en los programas del curso):

$$\mathbf{m} = v^T \mathbf{y}_k, \quad n_j = \frac{2}{1 + e^{-(m_j - c_j)/a_j}} - 1, \quad u_k = w^T \mathbf{n}$$

$$\frac{\partial n_j}{\partial m_j} = \frac{1 - n_j^2}{2a_j}, \quad \frac{\partial u}{\partial w} = \mathbf{n}, \quad \frac{\partial u}{\partial v} = \mathbf{y}_k (w^T D)$$

$$\frac{\partial u}{\partial c_j} = w_j \frac{n_j^2 - 1}{2a_j}, \quad \frac{\partial u}{\partial a_j} = w_j \frac{(n_j^2 - 1)(m_j - c_j)}{2a_j^2}, \quad \frac{\partial u}{\partial \mathbf{y}_k} = w^T D v^T, \quad D = \text{diag} \left(\frac{1 - n_j^2}{2a_j} \right)$$

2.5 Buenas prácticas de entrenamiento

- **Conjunto de posiciones iniciales:** se entrena simultáneamente con varias posiciones iniciales (en los programas del curso, cuatro), acumulando gradientes de todas las trayectorias; luego la red generaliza a otras posiciones.
- **Aprendizaje incremental:** como los humanos, la red aprende primero *tareas simples* (condiciones iniciales cercanas a un estado estable) y se amplía progresivamente el rango, reutilizando los pesos entrenados. Esencial para sistemas inestables en amplio rango de operación.

2.6 Tracking y realimentación parcial

Tracking / regulación en x^* . El controlador entrenado para estabilización, $u_k = N(\mathbf{x}_k)$, se adapta cambiando entradas y salida:

$$\boxed{u_k = u^* + N(\mathbf{x}_k - \mathbf{x}^*)} \quad \text{con la condición de equilibrio} \quad \boxed{\mathbf{x}^* = f(\mathbf{x}^*, u^*)}$$

($\mathbf{x}^*$ y u^* no se eligen de forma independiente).

Realimentación parcial de salida. Si no se mide todo el estado, el controlador usa sólo la(s) variable(s) medida(s) y, para recuperar la información dinámica, sus *valores pasados* (retardos z^{-1}) como entradas adicionales. El entrenamiento (dinámico) sigue igual, con $\frac{\partial u}{\partial \mathbf{y}}$ nulo en las componentes no medidas.

2.7 Integración de sistemas lineales y redes neuronales

- **Modelo no lineal = modelo lineal + red en paralelo:** $y_{k+1} = Ay_k + Bu_k + N(y_k, u_k)$. El modelo lineal aporta el conocimiento (parámetros); la red, entrenada con datos del sistema real, corrige la parte no lineal y amplía el rango de validez.
- **Controlador no lineal = controlador lineal + red en paralelo:** $u_k = u_k^{lineal} + N(y_k)$. El controlador lineal (diseñado sobre el modelo linealizado) funciona cerca del punto de operación; la red, entrenada con el modelo no lineal considerando al lineal, extiende y mejora el desempeño. (*Esta idea es la base del control lineal-difuso del truck-tráiler, Sección 7.*)
- **Controlador dinámico:** red recurrente $u_k = N(y_k, u_{k-1})$ entrenada considerando la dinámica del sistema.

3 Robots móviles tipo carro: modelamiento y control [Clase 10]

3.1 Tipos y modelo cinemático (modelo bicicleta)

Tipos: carro de 4 ruedas (delanteras dirección, traseras tracción), diferencial de 2 ruedas, y tipo tráiler (dos cuerpos articulados). El carro se reduce al **modelo bicicleta** (dos ruedas): rueda delantera = dirección (δ), rueda trasera = tracción; (x, y) es el punto central del eje trasero, ϕ la inclinación respecto al eje X , L la distancia entre ejes.

Supuestos: las ruedas no deslizan; baja velocidad ($v < 3$ m/s) $\Rightarrow$ **modelo cinemático** (sólo geometría; no intervienen masas ni fricciones). Ángulos antihorarios positivos.

$$\dot{x} = v \cos \phi, \quad \dot{y} = v \sin \phi, \quad \dot{\phi} = -\frac{v}{L} \tan \delta \quad (L \gg \Delta r)$$

Discretizando con avance por paso $\Delta r = v \Delta t$:

$$x_{k+1} = x_k + \Delta r \cos \phi_k, \quad y_{k+1} = y_k + \Delta r \sin \phi_k, \quad \phi_{k+1} = \phi_k - \frac{\Delta r}{L} \tan \delta_k$$

3.2 Estrategia de posicionamiento

Para alcanzar una posición deseada sobre una recta (p.ej. $x^* = 0$, $\phi^* = \pi/2$, avanzando luego hasta y^*), la estrategia es: **primero alcanzar la recta con la orientación correcta y después avanzar en línea recta** hasta la posición final (así se evita colisionar con obstáculos a los costados).

Idea clave: la señal de control δ *no depende de la coordenada a lo largo de la recta* (si la recta es vertical, δ no depende de y ; si es horizontal, no depende de x). El controlador sólo necesita la coordenada transversal y el ángulo: se reduce el problema a 2 variables. La convergencia debe ser *asintótica* (sin cruzar la recta oscilando).

3.3 Seguimiento de trayectorias: posición deseada perpendicular

En cada instante se traza una **perpendicular** desde la posición presente del robot a la trayectoria deseada; el punto de intersección define la posición deseada instantánea y la tangente en ese punto define la orientación deseada.

- **Recta a 45°** ($y = x - c$): $x^* = \frac{x+y}{2}$ (por geometría), $\phi^* = \pi/4$, $\delta^* = 0$.
- **Circunferencia de radio R** (centro conocido): la perpendicular pasa por el centro; por semejanza de triángulos se obtiene x^* (o y^*) y $\phi^* = \text{atan} \frac{50-x}{y}$ (para el círculo del ejemplo de clase), con

$$\delta^* = \text{atan} \frac{L}{R}$$

el ángulo de timón constante que corresponde a describir la circunferencia.

Medición/estimación de x y ϕ : sensores de distancia, cámaras, triangulación infrarroja, observadores y filtros de Kalman. (Problema de navegación con balizas b_1, b_2, b_3 .)

4 Control difuso: fundamentos y diseño [Clases 10 y 11 — material común]

4.1 Motivación y conceptos básicos

El conductor humano controla con lógica multivaluada basada en **acciones lingüísticas** (“girar el timón un poco a la izquierda”), aplicando reglas SI–ENTONCES aprendidas por experiencia, sin leyes matemáticas complejas. El control difuso imita ese mecanismo y sirve para estabilización, regulación y tracking.

- **Variable lingüística vs. numérica:** “la temperatura es alta” (lingüística) vs. “la temperatura es 32°C ” (numérica). Un valor numérico puede pertenecer a varios valores lingüísticos con distinto grado.
- **Función de pertenencia:** definida para cada valor lingüístico; da el grado de pertenencia de un valor numérico al valor lingüístico. Se definen con **traslape** entre particiones contiguas (sólo entre dos adyacentes) y de modo que la **suma de pertenencias sea 1**.
- **Reglas IF–THEN:** SI (*antecedente*: combinación de valores lingüísticos de las entradas) ENTONCES (*consecuente*: valor lingüístico de la salida). El conjunto completo es la **base de reglas**, núcleo del controlador, construida con la experiencia de un operador humano para *todas* las combinaciones posibles del antecedente.
- **Operadores lógicos:** AND = mínimo (o producto); OR = máximo (o suma acotada).

4.2 Proceso de diseño de un controlador difuso (receta general)

Pasos de diseño (aplican a cualquier problema del curso):

1. Determinar las **variables de entrada y salida** del controlador (típicamente: error de posición y velocidad; la salida es el actuador).
2. Determinar el **rango** de cada variable, de modo que el *valor deseado sea el punto medio* del rango (agregar margen adicional si es necesario; el rango de una velocidad se determina experimentalmente aplicando la entrada máxima, con margen del 20–30%).
3. **Particionar** cada rango en un número **impar** de particiones **simétricas**, con **traslape** entre contiguas.
4. Asignar un **valor lingüístico** a cada partición y definir su **función de pertenencia** (triangulares/trapezoidales).
5. Construir la **base de reglas** (experiencia humana; una regla por combinación del antecedente).
6. Calcular la salida en tiempo real por **inferencia MIN–MAX** y **defuzzificación** (centro de gravedad o Sugeno).

4.3 Ejemplo canónico: robot carro (recta vertical $x^* = 50$, $\phi^* = 90^\circ$)

Entradas x y ϕ , salida δ ; movimiento en retroceso a baja velocidad.

- $x \in [0, 100]$, punto medio $x^* = 50$; **5 particiones**: IZ, CI, CE, CD, DE.
- $\phi \in [-90^\circ, 270^\circ]$, punto medio $\phi^* = 90^\circ$; **7 particiones**: AD, HD, VD, VE, VI, HI, AI.
- $\delta \in [-30^\circ, 30^\circ]$, punto medio $\delta^* = 0$; **5 particiones**: NG, NM, ZE, PM, PG.
- Base de reglas: $5 \times 7 = 35$ reglas. Ejemplos: IF $x=CI$ AND $\phi=AD \rightarrow \delta=NG$; IF $x=IZ$ AND $\phi=VE \rightarrow \delta=NG$; IF $x=CD$ AND $\phi=AI \rightarrow \delta=PM$.

Inferencia MIN-MAX (Mamdani) — tres casos vistos en clase:

1. **Se activa 1 regla** ($x = 62$, $\phi = -22^\circ$): $x=CD$ (0.62), $\phi=HD$ (0.86), luego $\delta=PG$ con $\min(0.62, 0.86) = 0.62$; centro de gravedad del área de PG recortada a 0.62 $\Rightarrow \delta = 22.5^\circ$.
2. **Se activan 2 reglas** ($x = 55$, $\phi = -22^\circ$): $(CE, HD) \rightarrow NG$ con 0.08; $(CD, HD) \rightarrow PG$ con 0.36; COG de ambas áreas $\Rightarrow \delta = 20.2^\circ$.
3. **Se activan 4 reglas** ($x = 55$, $\phi = 5^\circ$): $(CE, HD) \rightarrow NG$ 0.08; $(CE, VD) \rightarrow PG$ 0.08; $(CD, HD) \rightarrow PG$ 0.36; $(CD, VD) \rightarrow PM$ 0.36; COG $\Rightarrow \delta = 12.6^\circ$.

Defuzzificación por método de Sugeno: a cada partición de la salida se le asigna un *valor representativo* (NG= -30, NM= -10, ZE= 0, PM= 10, PG= 30) y

$$\delta = \frac{\sum_i \mu_i f_i}{\sum_i \mu_i} \quad (\text{caso 3: } \delta = \frac{-30(0.08)+30(0.08)+30(0.36)+10(0.36)}{0.08+0.08+0.36+0.36} = 16.36^\circ)$$

Más simple de calcular que el COG y de resultado similar.

Extensiones (sin rediseñar):

- **Otra posición deseada x^* :** corrimiento de la entrada: $x \leftarrow x - (x^* - 50)$.
- **Recta horizontal ($y^*, \phi^* = 0$):** el mismo controlador con entradas y y ϕ .

Implementación en Matlab: (1) búsqueda de las particiones activas de x , que produce **ValueX** (pares partición-pertenencia); (2) ídem para ϕ , que produce **ValueP**; (3) con ambas se consulta la base de reglas y se obtiene **ValueD** (particiones de δ activas con sus grados); (4) se calculan las áreas recortadas y el centro de gravedad.

4.4 Segundo ejemplo: posición de un motor DC con tornillo sinfín

Masa m desplazada por un motor DC con tornillo sinfín; objetivo $x \rightarrow x^* = 0$; voltaje positivo desplaza a la derecha. Modelo: $x_{k+1} = Ax_k + Bu_k + WF_{s_k}$ con $\mathbf{x} = [x \ \dot{x}]^T$ y fricción seca F_s .

- Entradas del controlador: x y $\dot{x}$ (imprescindible); salida: V . Rangos con margen: $x \in [-0.75, 0.75]$, $\dot{x} \in [-0.85, 0.85]$ (experimental + 20-30%), $V \in [-30, 30]$.
- Particiones: x : MI, IZ, CE, DE, MD; $\dot{x}$: RI, IZ, ZE, DE, RD; V : NE, ZE, PO.
- Reglas por razonamiento físico, p. ej.: IF $x=MD$ AND $\dot{x}=RD \rightarrow V=NE$ (muy a la derecha y alejándose rápido: frenar y regresar); IF $x=CE$ AND $\dot{x}=RI \rightarrow V=PO$ (en el centro pero yéndose rápido a la izquierda: frenar).
- Otras posiciones deseadas: corrimiento del rango de x .
- **Robustez:** analizar el efecto de la fricción seca (produce *error en estado estacionario*).

5 Control difuso del robot truck-tráiler

[Clase 12]

Robot de dos cuerpos articulados (truck L_1 + tráiler L_2), en retroceso, sin deslizamiento, $v < 3$ m/s. Aparece la variable crítica $\theta_{12} = \theta_1 - \theta_2$ y el riesgo de **jack-knife** ($\theta_{12} \rightarrow \pm 90^\circ$).

- **Entradas del controlador difuso:** x (7 particiones), θ_2 (7), θ_{12} (3: NE-ZE-PO). **Salida:** δ (7 particiones). En la práctica bastan menos particiones (5 para x , 5 para θ_2 , 3 para δ).
- **Base de reglas tridimensional:** se construyen *tres matrices* 7×7 (una por cada valor de θ_{12}):
 - $\theta_{12} = \text{ZERO}$: se construye analizando caso por caso (como el robot carro).
 - $\theta_{12} = \text{NEG}$: *toda* la matriz $\rightarrow \delta$ negativo máximo (partición 1).
 - $\theta_{12} = \text{POS}$: *toda* la matriz $\rightarrow \delta$ positivo máximo (partición 7).
- **Justificación (análisis dinámico):** hay relación inversa entre $\dot{\theta}_{12}$ y δ . Si θ_{12} es negativo (cerca de -90°), $\dot{\theta}_{12}$ debe ser positivo máximo $\Rightarrow \delta$ negativo máximo; si θ_{12} es positivo, lo contrario. **Prioridad absoluta: evitar el jack-knife;** sólo cuando $\theta_{12} \approx 0$ se atiende el posicionamiento.
- La inferencia es la misma (fuzzificar las 3 entradas, mínimo por regla, defuzzificación por COG). Ejemplo de clase: $x = 30$, $\theta_2 = 88^\circ$, $\theta_{12} = 48^\circ$ activa reglas con mínimos 0.5 y 0.4.
- Las funciones de pertenencia de θ_{12} pueden modificarse (ancho de la partición ZERO) para ajustar el compromiso velocidad de convergencia vs. magnitud de θ_{12} (ver Sección 7).

6 Redes neuro-difusas

[Clase 12]

6.1 Motivación y tipos

Redes neuronales	Lógica difusa
Procesan datos; pueden aprender	Procesa experiencia humana
Óptimas para optimización	Estructura clara mediante reglas
Estructura oscura (caja negra)	No requiere modelo del sistema

La red neuro-difusa combina ambas: *lógica difusa implementada mediante una red neuronal* (estructura clara + capacidad de aprender).

Tres tipos (según el consecuente; el antecedente es igual en los tres):

- **Tipo 1:** el consecuente es un *valor numérico*: IF $x_1=\text{IZ}$ AND $x_2=\text{VE}$ $\rightarrow u = 15$.
- **Tipo 2:** el consecuente es una *función de las entradas*: $u = g(x_1, x_2)$.
- **Tipo 3:** el consecuente es un *valor lingüístico*: $u = \text{NB}$ (con su función de pertenencia).

6.2 Estructura (red Tipo 1) y entrenamiento

Capas: (1) entradas numéricas $\rightarrow$ (2) funciones de pertenencia parametrizadas (centro, pendiente) $\rightarrow$ (3) reglas: AND como **producto** de las pertenencias del antecedente $\rightarrow$ (4) **normalización** $\bar{\mu}_i = \mu_i / \sum \mu_i \rightarrow$ (5) salida = $\sum_i \bar{\mu}_i f_i$ con f_i el valor representativo del consecuente de cada regla.

Proceso de diseño: (1) construir el controlador fuzzy clásico; (2) representarlo como red neuro-difusa (al inicio trabaja *exactamente igual* que el fuzzy); (3) *si se desea*, entrenarla (DBP) para modificar los valores representativos f_i del consecuente o los parámetros (centro, pendiente) de las funciones de pertenencia. **Importante:** considerar el factor de normalización $1 / \sum \mu_i$ en el cálculo de las derivadas totales.

En el Tipo 2 se entrenan los coeficientes de las funciones $g(x_1, x_2)$ del consecuente. Aplicación de referencia: *Identification and Control of Nonlinear Vehicle Dynamics Using Neural Networks* (A. Morán). Trabajo del curso: controlador neuro-difuso del truck-tráiler a partir del fuzzy (`fuzzytrailerxbueno.m`, `neurofuzzy1.m`), ajustando el factor de salida, sin entrenamiento, verificando varias posiciones iniciales.

6.3 Formación de robots (aplicación)

Líder-seguidor: el líder apunta a la posición final deseada; la posición deseada del seguidor es la posición *presente* del líder (con varios seguidores, cada uno sigue al anterior). Cada robot usa su propio controlador de posicionamiento.

7 Control lineal-difuso del truck-tráiler [Clase 13 — paper Morán & Nagai]

Es la síntesis de las ideas del curso: *controladores lineales simples integrados mediante lógica difusa*.

7.1 Modelo y estrategia

Estados $\mathbf{x} = [y \ \theta_2 \ \theta_{12}]^T$, entrada $u = \tan \delta$, v constante (retroceso):

$$\dot{y} = v \cos \theta_{12} \sin \theta_2, \quad \dot{\theta}_2 = -\frac{v}{L_2} \sin \theta_{12}, \quad \dot{\theta}_{12} = \frac{v}{L_2} \sin \theta_{12} - \frac{v}{L_1} \tan \delta$$

La coordenada x **no** se incluye (al converger a $y = 0$, x es proporcional al tiempo). Estrategia: alcanzar la recta $y^* = 0$ con $\theta_2^* = 0$, $\theta_{12}^* = 0$ y avanzar recto entre los obstáculos $\Rightarrow$ el posicionamiento se vuelve un problema de *estabilización* de 3 variables.

7.2 Diseño: linealización + integración difusa

Linealizando alrededor de $\theta_2^* = 0$, $\theta_{12}^* = 0$ ($\sin \theta \approx \theta$, $\cos \theta \approx 1$):

$$\begin{bmatrix} \dot{y} \\ \dot{\theta}_2 \\ \dot{\theta}_{12} \end{bmatrix} = \begin{bmatrix} 0 & v & 0 \\ 0 & 0 & -v/L_2 \\ 0 & 0 & v/L_2 \end{bmatrix} \begin{bmatrix} y \\ \theta_2 \\ \theta_{12} \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ -v/L_1 \end{bmatrix} \tan \delta \quad \Rightarrow \quad \tan \delta = -k_1 y - k_2 \theta_2 - k_3 \theta_{12}$$

válida sólo cerca de $\theta_{12} = 0$. Para cubrir todo el rango ($-90^\circ, 90^\circ$) se particiona θ_{12} en NB/ZE/PB y se integran tres controladores como suma ponderada por las pertenencias:

$$\text{IF } \theta_{12} = \text{PB} \rightarrow \delta = \delta_{\max}; \quad \text{IF } \theta_{12} = \text{ZE} \rightarrow \delta = \text{ley lineal}; \quad \text{IF } \theta_{12} = \text{NB} \rightarrow \delta = \delta_{\min}$$

(los extremos evitan el jack-knife: relación inversa entre $\dot{\theta}_{12}$ y $\tan \delta$). Para otras posiciones deseadas:

$$\tan \delta - \tan \delta^* = -k_1(y - y^*) - k_2(\theta_2 - \theta_2^*) - k_3(\theta_{12} - \theta_{12}^*)$$

con valores deseados *coherentes y físicamente realizables*.

7.3 Seguimiento de trayectorias (posición deseada perpendicular)

Se traza la perpendicular desde la posición presente (x, y) a la trayectoria deseada: la intersección da y^* y la tangente da θ_2^* ; con las ecuaciones cinemáticas se obtienen θ_{12}^* y δ^* .

- **Recta** $y = ax + b$ (inclinación α): $y^* = \frac{ax + a^2y + b}{1 + a^2}$, $\theta_2^* = \alpha$, $\theta_{12}^* = 0$, $\delta^* = 0$.

- **Circunferencia** (centro (x_c, y_c) , radio R): la perpendicular pasa por el centro;

$$y^* = \frac{R(y - y_c)}{\sqrt{(x - x_c)^2 + (y - y_c)^2}} + y_c, \quad \theta_2^* = \text{atan} \frac{x_c - x}{y - y_c}$$

$$\boxed{\theta_{12}^* = \text{atan} \frac{L_2}{R}}, \quad \boxed{\delta^* = \text{atan} \frac{L_1}{\sqrt{L_2^2 + R^2}}}$$

(θ_{12}^* y δ^* son *constantes*, como corresponde a una trayectoria circular; comparar con $\delta^* = \text{atan}(L/R)$ del carro simple).

7.4 Efecto del ancho de las particiones de θ_{12}

Una partición ZE **más ancha** (extremos NB/PB con menor pertenencia) impone menos restricción sobre θ_{12} : el robot converge **más rápido** y con menor radio de giro, a costa de valores **mayores** de θ_{12} y δ (sin llegar al jack-knife). Una ZE angosta da la conducta opuesta (más conservadora). Es el parámetro de sintonía del compromiso rapidez vs. seguridad.

8 Formulario / hoja de repaso rápido

Redes y entrenamiento

$$\text{Sigmoide bipolar: } n = \frac{2}{1 + e^{-(m-c)/a}} - 1, \quad \frac{\partial n}{\partial m} = \frac{1 - n^2}{2a}$$

$$\frac{\partial n}{\partial c} = \frac{n^2 - 1}{2a}, \quad \frac{\partial n}{\partial a} = \frac{(n^2 - 1)(m - c)}{2a^2}$$

$$\text{Costo: } J = \frac{1}{2} \sum_{k=1}^N (\mathbf{x}_k - \mathbf{x}^*)^T Q (\mathbf{x}_k - \mathbf{x}^*), \quad \frac{\partial J}{\partial \theta} = \sum_k (\mathbf{x}_k - \mathbf{x}^*)^T Q \frac{\partial \mathbf{x}_k}{\partial \theta}$$

$$\text{DBP: } \frac{\partial \mathbf{x}_{k+1}}{\partial \theta} = \frac{\partial \mathbf{x}_{k+1}}{\partial u_k} \frac{\partial u_k}{\partial \theta} + \left[\underbrace{\frac{\partial \mathbf{x}_{k+1}}{\partial \mathbf{x}_k}}_{\text{modelo}} + \underbrace{\frac{\partial \mathbf{x}_{k+1}}{\partial u_k}}_{\text{modelo}} \underbrace{\frac{\partial u_k}{\partial \mathbf{x}_k}}_{\text{controlador}} \right] \frac{\partial \mathbf{x}_k}{\partial \theta}, \quad \frac{\partial \mathbf{x}_0}{\partial \theta} = 0$$

$$\text{Tracking: } u_k = u^* + N(\mathbf{x}_k - \mathbf{x}^*) \text{ con } \mathbf{x}^* = f(\mathbf{x}^*, u^*)$$

Robot carro

$$\dot{x} = v \cos \phi, \quad \dot{y} = v \sin \phi, \quad \dot{\phi} = -\frac{v}{L} \tan \delta; \quad y_{k+1} = y_k + \Delta r \sin \phi_k, \quad \phi_{k+1} = \phi_k - \frac{\Delta r}{L} u_k$$

$$\frac{\partial \mathbf{x}_{k+1}}{\partial \mathbf{x}_k} = \begin{bmatrix} 1 & \Delta r \cos \phi_k \\ 0 & 1 \end{bmatrix}, \quad \frac{\partial \mathbf{x}_{k+1}}{\partial u_k} = \begin{bmatrix} 0 \\ -\Delta r / L \end{bmatrix}, \quad \delta_{\text{círculo}}^* = \text{atan} \frac{L}{R}$$

Truck-tráiler

$$\dot{y} = v \cos \theta_{12} \sin \theta_2, \quad \dot{\theta}_2 = -\frac{v}{L_2} \sin \theta_{12}, \quad \dot{\theta}_{12} = \frac{v}{L_2} \sin \theta_{12} - \frac{v}{L_1} \tan \delta$$

$$\tan \delta = -k_1 y - k_2 \theta_2 - k_3 \theta_{12} \quad (\theta_{12} \approx 0); \quad \theta_{12}^* = \text{atan} \frac{L_2}{R}, \quad \delta^* = \text{atan} \frac{L_1}{\sqrt{L_2^2 + R^2}}$$

Control difuso

- Diseño: entradas/salida $\rightarrow$ rangos (deseado al centro) $\rightarrow$ particiones impares simétricas con traslape (suma de pertenencias = 1) $\rightarrow$ valores lingüísticos y MFs $\rightarrow$ base de reglas (todas las combinaciones) $\rightarrow$ inferencia.
- Inferencia Mamdani: fuzzificar $\rightarrow$ AND = mínimo por regla $\rightarrow$ agregación por máximo $\rightarrow$ COG del área recortada.
- Sugeno: $u = \frac{\sum_i \mu_i f_i}{\sum_i \mu_i}$ con f_i valores representativos.
- Otra posición deseada: corrimiento de la entrada ($x \leftarrow x - (x^* - x_{\text{centro}})$).
- Red neuro-difusa Tipo 1: pertenencias $\rightarrow$ producto $\rightarrow$ normalización $\rightarrow \sum \bar{\mu}_i f_i$; entrenable (f_i o parámetros de MFs, considerando la normalización en las derivadas).

Mapa de temas $\rightarrow$ Examen Final 2025-1

Pregunta	Tema	Sección de este resumen
P1	Control lineal-difuso truck-tráiler, y lejanos	7 (+ 5)
P2	Diseño de controlador difuso (motor CC) + red neuro-difusa	4 + 6
P3	DBP: costo, ecuación recursiva, matrices del robot carro	2.4 + 3.1 + 8
P4	Medición parcial, aprendizaje incremental, tracking	2.5 + 2.6
P5	Programa Matlab del DBP (robot carro)	2.4 + 3 + 8

Resumen elaborado íntegramente a partir del material del curso (Clases 7, 9, 10, 11, 12 y 13).